Definición de Infraestructura para procesameinto de datos con Spark

Sergio Andrés Guevara

Samuel David Rojas

Universidad del Rosario

Escuela de Ciencias e Ingeniería

Matemáticas Aplicadas y Ciencias de la Computación

Big Data

Bogotá, D.C.

2026

Índice

1. Topología Clúster EMR **2**

2. Parámetros de configuración de Apache Spark **2**

2.1. Derivación de parámetros por nodo (m5.xlarge: 4 vCPU / 16 GiB RAM) 2

2.2. Configuración completa - spark-deafults.conf 2

2.3. Ajuste de parámetros para experimentos de escalabilidad horizontal 3

3. Estructura del Bucket S3 del Proyecto **3**

4. Configuración de Spark Structured Streaming **4**

4.1. Medición de latencia extremo a extremo 5

5. Aprovisionamiento del Clúster **5**

5.1. AWS CLI 5

5.2. Archivo emr-spark-config.json 6

6. Región AWS y Configuración de Red **6**

1. Topología Clúster EMR

Rol	Tipo EC2	vCPU	RAM(GiB)	Almac. (GiB)	Cantidad
Master	m5.xlarge	4	16	64 EBS	1
Core	m5.xlarge	4	16	64 EBS	2 (minimo)

La justificación de la instancia m5.xlarge como unidad base del clúster atiende a los siguientes criterios:

- Relación vCPU/RAM adecuada para cargas Spark mixtas (entrenamiento ML + Structured Streaming).
- Costo por hora on-demand de $0.192 USD, lo que permite experimentos de múltiples horas dentro de un presupuesto académico razonable.
- Disponibilidad garantizada en la región us-east-1, donde se aloja el bucket público de CSE-CIC-IDS2018.
- Escalabilidad homogénea: todos los nodos comparten el mismo tipo de instancia, eliminando asimetrías en la distribución de tareas Spark.

2. Parámetros de configuración de Apache Spark

Los parámetros expuestos a continuación son los determinantes del comportamiento del motor distribuido. Se derivan de las especificaciones de hardware del nodo m5.xlarge siguiendo las heurísticas estándar de dimensionamiento de Spark.

2.1. Derivación de parámetros por nodo (m5.xlarge: 4 vCPU / 16 GiB RAM)

Decisión de dimensionamiento	Valor	Justificación
Cores por ejecutor	2	Equilibrio entre paralelismo intra-tarea y contención de I/O
Ejecutores por nodo core	1	4 vCPU - 1 (YARN NM) = 3 disponibles; 1 ejecutor × 2 cores + 1 libre para OS
RAM por ejecutor	6 GiB	16 GiB - 6 GiB (YARN NM + OS) = 12 GiB; 6 GiB por ejecutor + overhead
spark.memory.fraction	0.75	Mayor fracción para entrenamiento RF con grandes vectores de características
spark.memory.storageFraction	0.40	Caché de DataFrames de entrenamiento que se reutilizan en múltiples epochs

2.2. Configuración completa - spark-deafults.conf

```
# Resources per executor
spark.executor.cores                2
spark.executor.memory               6g
spark.executor.memoryOverhead       1024

# Driver (Master node m5.xlarge)
spark.driver.cores                  2
spark.driver.memory                 6g
```

```
spark.driver.memoryOverhead          512

# Instances (base cluster: 1 master + 2 core)
spark.executor.instances             2

# Parallelism
# Rule: 2-3x total available cores
# Base (2 core nodes x 1 executor x 2 cores = 4 cores) -> 16 partitions
spark.default.parallelism            16
spark.sql.shuffle.partitions         16

# I/O and serialization
spark.serializer                     org.apache.spark.serializer.KryoSerializer
spark.kryoserializer.buffer.max      256m
spark.sql.files.maxPartitionBytes    134217728
spark.sql.files.openCostInBytes      4194304

# Fault tolerance
spark.task.maxFailures               4
spark.speculation                    true
spark.speculation.multiplier         1.5

# S3 (direct read/write)
spark.hadoop.fs.s3a.impl             org.apache.hadoop.fs.s3a.S3AFileSystem
spark.hadoop.fs.s3a.fast.upload      true
spark.hadoop.fs.s3a.multipart.size   67108864
spark.hadoop.fs.s3a.connection.maximum 100
```

2.3. Ajuste de parámetros para experimentos de escalabilidad horizontal

Parámetro	2 nodos core	3 nodos core	4 nodos core	5 nodos core
spark.executor.instances	2	3	4	5
spark.default.parallelism	16	24	32	40
spark.sql.shuffle.partitions	16	24	32	40
Total cores disponibles	4	6	8	10
RAM ejecutor total	12 GiB	18 GiB	24 GiB	30 GiB

3. Estructura del Bucket S3 del Proyecto

Todos los artefactos del proyecto se almacenan en un bucket S3 dedicado (denominado s3://proyecto-anomalias-emr/ a efectos de esta especificación). La estructura de directorios garantiza la separación entre datos de entrada, modelos entrenados, puntos de control de streaming y resultados de evaluación.

```
s3://proyecto-anomalias-emr/
├── data/
│   ├── cic-ids2017/
│   │   ├── train/
│   │   │   ├── Monday-WorkingHours.pcap_ISCX.csv
│   │   │   ├── Wednesday-WorkingHours.pcap_ISCX.csv
│   │   │   └── Friday-WorkingHours.pcap_ISCX.csv
│   │   └── test/
│   │       ├── Tuesday-WorkingHours.pcap_ISCX.csv
│   │       └── Thursday-WorkingHours.pcap_ISCX.csv
│   └── csc-cic-ids2018/
│       └── (reference: s3://cse-cic-ids2018/Processed Traffic Data/)
├── models/
│   ├── random_forest_pipeline/
│   └── kmeans_pipeline/
├── checkpoints/
│   ├── rf_streaming/
│   └── kmeans_streaming/
├── results/
│   ├── batch_evaluation/
│   ├── streaming_metrics/
│   └── scalability/
├── logs/
│   └── spark_event_logs/
└── scripts/
    ├── 01_ingestion.py
    ├── 02_train_rf.py
    ├── 03_train_kmeans.py
    ├── 04_evaluate_batch.py
    └── 05_streaming_demo.py
```

4. Configuración de Spark Structured Streaming

Los parámetros de streaming definen el comportamiento del pipeline near real-time durante la fase de demostración y son determinantes para las métricas de latencia y throughput reportadas.

Parámetro	Valor configurado	Descripción
Trigger interval	processingTime = '10s'	Duración de cada micro-batch. Define la resolución temporal de la detección
Checkpoint location	s3://proyecto-anomalias-emr/checkpoints/	Almacenamiento de offsets y estado del query para tolerancia a fallos.
Output mode	append	Solo se escriben las filas nuevas de cada micro-batch.
maxFilesPerTrigger	5	Número máximo de archivos CSV consumidos por micro-batch desde S3.
Watermark (event time)	30 seconds	Tolerancia para datos tardíos en el stream.
sink	console + S3 parquet	Visualización en tiempo real y persistencia de predicciones.
spark.sql.streaming.metricsEnabled	true	Expone throughput y latencia a Spark UI y Ganglia EMR.

4.1. Medición de latencia extremo a extremo

La latencia de detección se define como el tiempo transcurrido entre la marca temporal de llegada del primer registro de un micro-batch al pipeline de Spark (t_{ingest}) y el momento en que la predicción queda disponible en el sink S3 ($t_{predict}$). Esta medición se realiza mediante los campos batchId, numInputRows, triggerExecution e inputRowsPerSecond expuestos por StreamingQueryListener, implementado como observer en el script 05_streaming_demo.py.

```
# Fragmento ilustrativo, StreamingQueryListener
class LatencyListener(StreamingQueryListener):
    def onQueryProgress(self, event):
        prog = event.progress
        print({
            'batchId'           : prog.batchId,
            'inputRows'         : prog.numInputRows,
            'inputRowsPerSec'   : prog.inputRowsPerSecond,
            'processedRowsPerSec': prog.processedRowsPerSecond,
            'triggerExecMs'     : prog.durationMs['triggerExecution'],
            'addBatchMs'        : prog.durationMs['addBatch'],
        })
spark.streams.addListener(LatencyListener())
```

5. Aprovisionamiento del Clúster

5.1. AWS CLI

El siguiente comando de AWS CLI reproduce exactamente la configuración del clúster base (1 master + 2 core nodes). Debe ejecutarse desde una sesión con credenciales IAM con permisos EMR, EC2 y S3.

```
aws emr create-cluster \
  --name "ProyectoAnomalias-BD2026" \
  --release-label emr-7.1.0 \
  --region us-east-1 \
  --applications \
      Name=Spark \
      Name=Hadoop \
      Name=JupyterEnterpriseGateway \
  --instance-groups \
      InstanceGroupType=MASTER,InstanceType=m5.xlarge,InstanceCount=1 \
      InstanceGroupType=CORE,InstanceType=m5.xlarge,InstanceCount=2 \
  --ec2-attributes \
      KeyName=<EC2_KEY_PAIR>,SubnetId=<SUBNET_ID> \
  --service-role EMR_DefaultRole \
  --ec2-attributes \
      InstanceProfile=EMR_EC2_DefaultRole \
  --log-uri s3://proyecto-anomalias-emr/logs/emr/ \
  --bootstrap-actions \
      Path=s3://proyecto-anomalias-emr/scripts/bootstrap.sh \
  --configurations file://emr-spark-config.json \
  --enable-debugging \
  --auto-terminate
```

5.2. Archivo emr-spark-config.json

```
  [
{
  "Classification": "spark-defaults",
  "Properties": {
    "spark.executor.cores":                  "2",
    "spark.executor.memory":                 "6g",
    "spark.executor.memoryOverhead":         "1024",
    "spark.driver.memory":                   "6g",
    "spark.driver.cores":                    "2",
    "spark.executor.instances":              "2",
    "spark.default.parallelism":             "16",
    "spark.sql.shuffle.partitions":          "16",
    "spark.serializer":                      "org.apache.spark.serializer.KryoSerializer
 ",
    "spark.kryoserializer.buffer.max":       "256m",
    "spark.memory.fraction":                 "0.75",
    "spark.memory.storageFraction":          "0.40",
    "spark.task.maxFailures":                "4",
    "spark.sql.streaming.metricsEnabled":    "true",
    "spark.hadoop.fs.s3a.fast.upload":       "true"
  }
},
{
  "Classification": "spark-env",
  "Configurations": [{
    "Classification": "export",
    "Properties": { "PYSPARK_PYTHON": "/usr/bin/python3" }
  }]
 }
]
```

6. Región AWS y Configuración de Red

u-east-1 (N. virginia) Co-localización con bucket público de CSE-CIC-IDS2018

Parámetro de red	Valor	Justificación
VPC / subnet	Default VPC (pública)	Acceso simplificado para el contexto académico.
Grupos de seguridad	ElasticMapReduce-master / -slave	Grupos EMR por defecto; permiten comunicación interna entre nodos y acceso SSH desde IP.
Par de claves EC2	Generado específicamente para el proyecto	Permite acceso SSH al nodo master para inspección y depuración manual.
Transferencia S3 $\Longleftrightarrow$ EMR	Sin costo adicional	Dentro de la misma región; VPC Gateway Endpoint para S3 configurado por defecto en EMR.

Referencias

[1] Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., ... & Xin, R. (2016). *MLlib: Machine learning in Apache Spark*. Journal of Machine Learning Research, 17(1), 1235–1241.

[2] Armbrust, M., Das, T., Torres, J., Yavuz, B., Zhu, S., Xin, R., ... & Zaharia, M. (2018). *Structured streaming: A declarative API for real-time applications in Apache Spark*. Proceedings of the ACM SIGMOD International Conference on Management of Data.

[3] AWS.(2026). *Apache Spark on Amazon EMR*. Documentation of Apache Spark EMR on. https://aws.amazon.com/emr/features/spark/